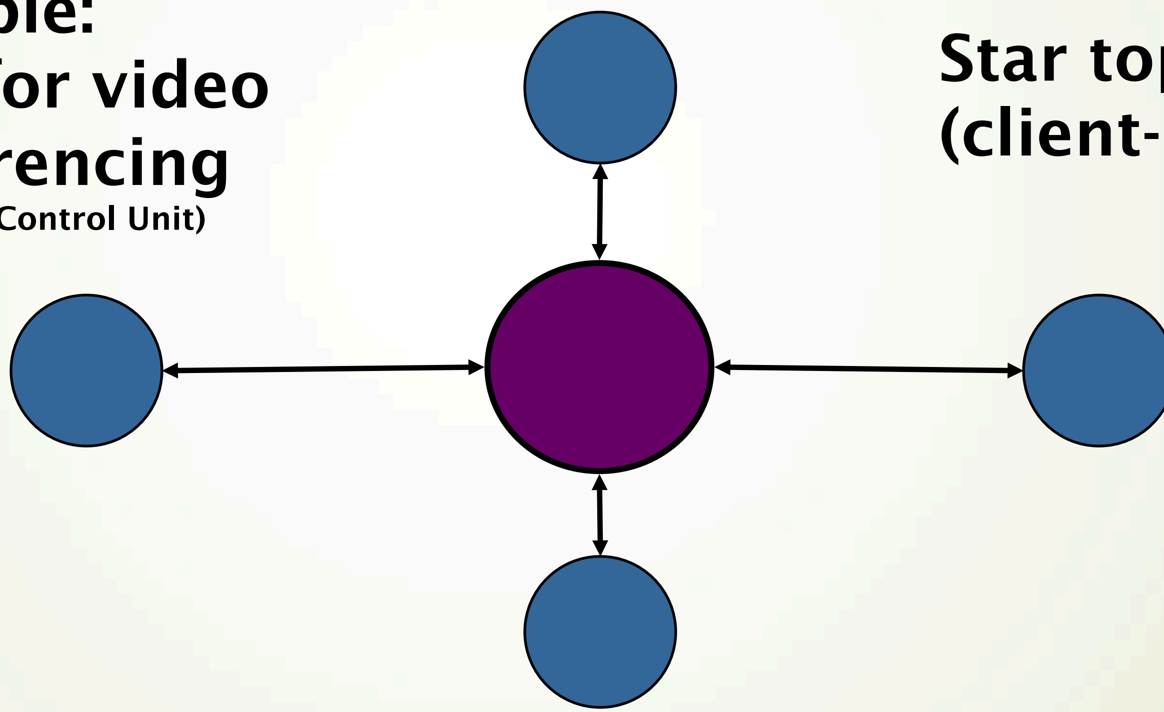
Comm. Model: Multicast (3/3)

Example:
MCU for video conferencing
(Multipoint Control Unit)

Star topology (client-server)



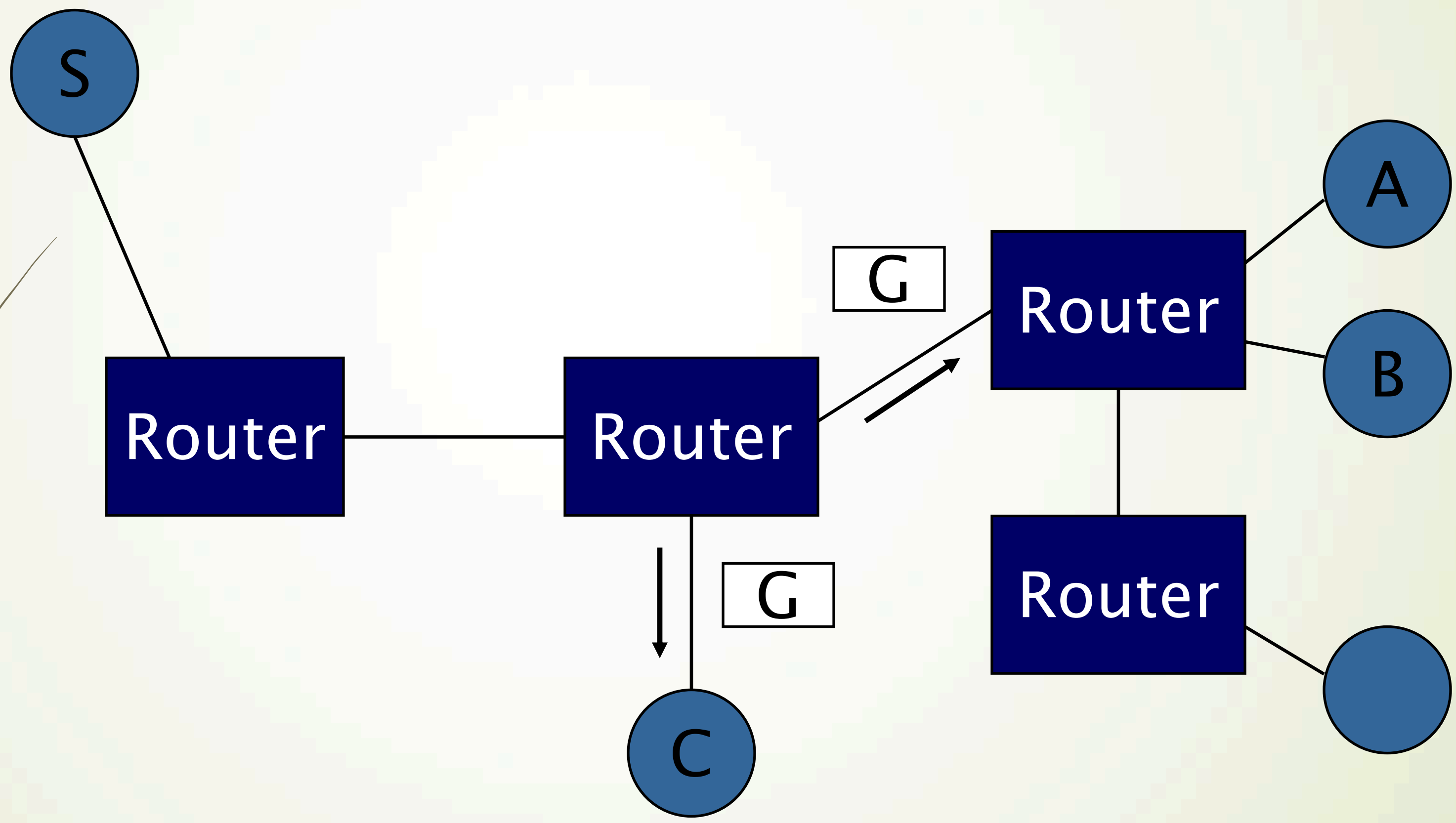
Media applications:
Many-to-Many

Tradeoff Between Mesh and Star Topologies (in App Layer)

- Mesh
  - N-1 connections at each client
  - $(N \times (N-1))/2$ connections total
  - Not scalable!
- Star
  - 1 connection per client
  - Server resources become a bottleneck
  - Single point of failure

Tradeoff is common in multimedia networking!

IP Multicast Comes to Rescue



BTW, you can only do IP multicast with UDP packets…Why?

Current Status of Multicast

- IP multicast has been standardized long ago and is implemented in almost all major routers, but
  - Technical and non-technical reasons hinder its adoption in much of the Internet
  - More precisely, it failed….
- Because of the unavailability of IP multicast, still too many applications use application-level multicast
- **Can you think of some reasons?**

What I can Think of….

- Who assigns a multicast address?
- Who pays for multicast traffic?
- How to inter-operate between protocols?
- How can we prevent DoS?

- Next question is: What do unicast transport protocols offer?

Agenda

- Media Streaming
- Video Quality Metrics
- Network Communication Models
- Different Transport Protocols
- Multimedia Friendly Internet

Recap: What Is the Difference Between TCP and UDP?

- TCP
  - connection oriented
  - packet ordering
  - reliability
  - congestion control
- UDP
  - just send!

Conventional Wisdom

- Continuous media uses UDP
  - Retransmission may not be useful ← Why?
  - Congestion control makes throughput unpredictable ← Even worse than low throughput
  - Multicast + TCP has problems

The Fact is UDP Alone is Not Enough

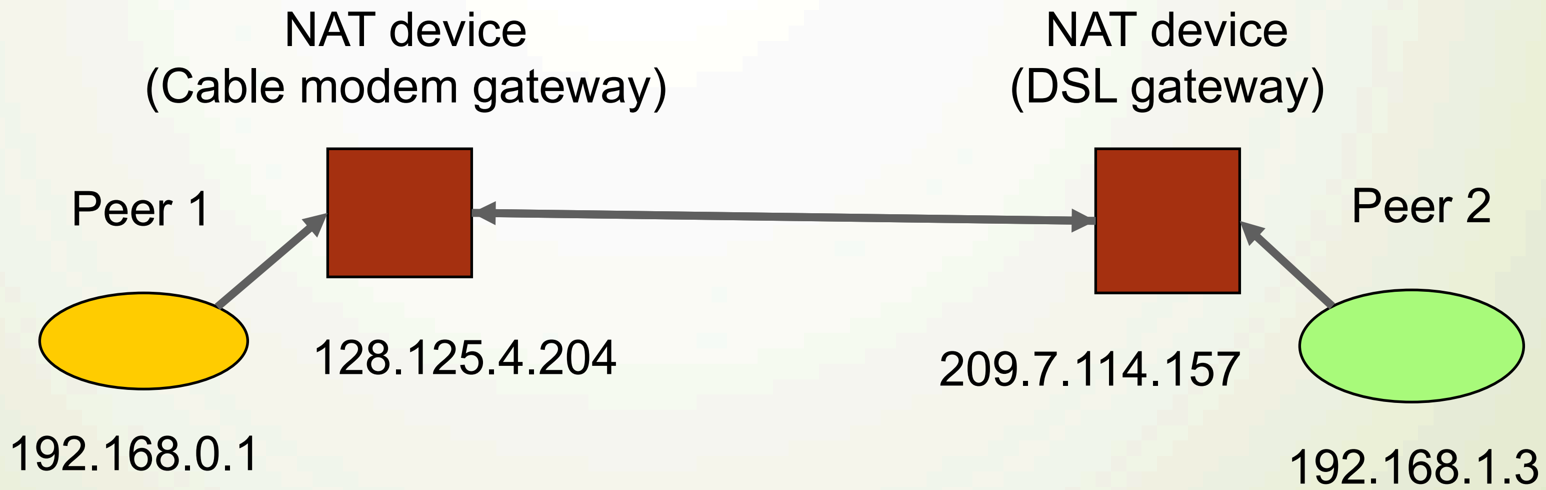
- Receiver will run into these questions:
  - Who sent this packet?
  - How do I interpret this packet?
  - When was this packet generated?
  - Which packets come first?
  - Is this packet important?
  - Should I ask for retransmission?

Solution: Application-Level Framing

- Expose details to applications
- Let application decide what to do with a packet, not transport protocol
- Example: RTP for push-based streaming ← More about this next week

UDP and NAT Traversal

- Many residential computers use network address translation (NAT)



NAT Traversal (1/2)

- Connection reversal
  - When only the server is behind the NAT
- UPnP and ICE protocols
  - Configure your gateway automatically
- TURN
  - Concatenate two connections into a single one
- STUN
  - STUN protocol (RFC 5389) and server to find my public IP address

NAT Traversal (2/2)

- UDP hole punching
  - Third party host is used to initially establish correct state in the routers
  - State periodically expires: keep-alive messages may be needed in the absence of traffic
- But only sub-80% successful rate!
  - Modern workaround: TCP (a.k.a. DASH) streaming

Agenda

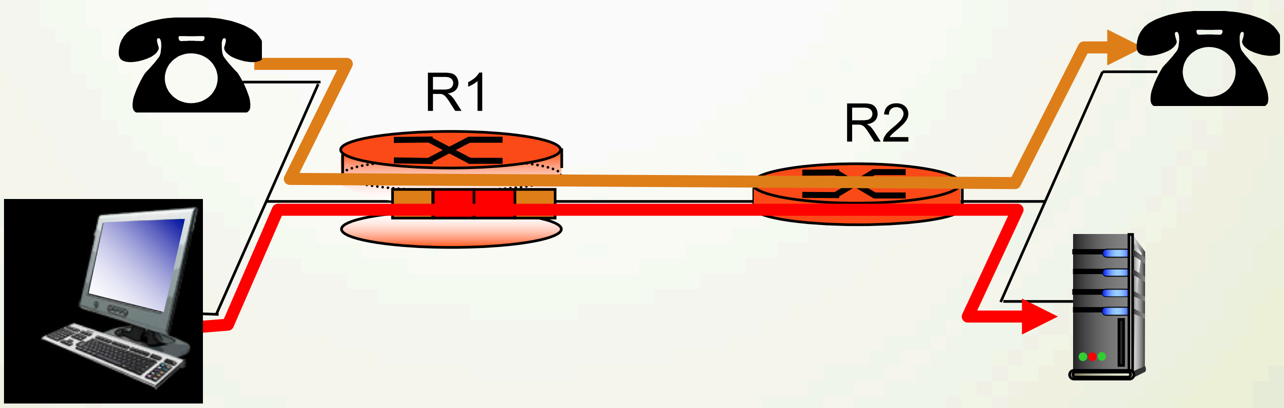
- Media Streaming
- Video Quality Metrics
- Network Communication Models
- Different Transport Protocols
- Multimedia Friendly Internet

Network Supports

Approach	Granularity	Guarantee	Mechanisms	Complex	Deployed?
Making best of best effort service	All traffic treated equally	None or soft	No network support (all at application)	low	everywhere
Differentiated service	Traffic "class"	None of soft	Packet market, scheduling, policing.	med	some
Per-connection QoS	Per-connection flow	Soft or hard after flow admitted	Packet market, scheduling, policing, call admission	high	little to none

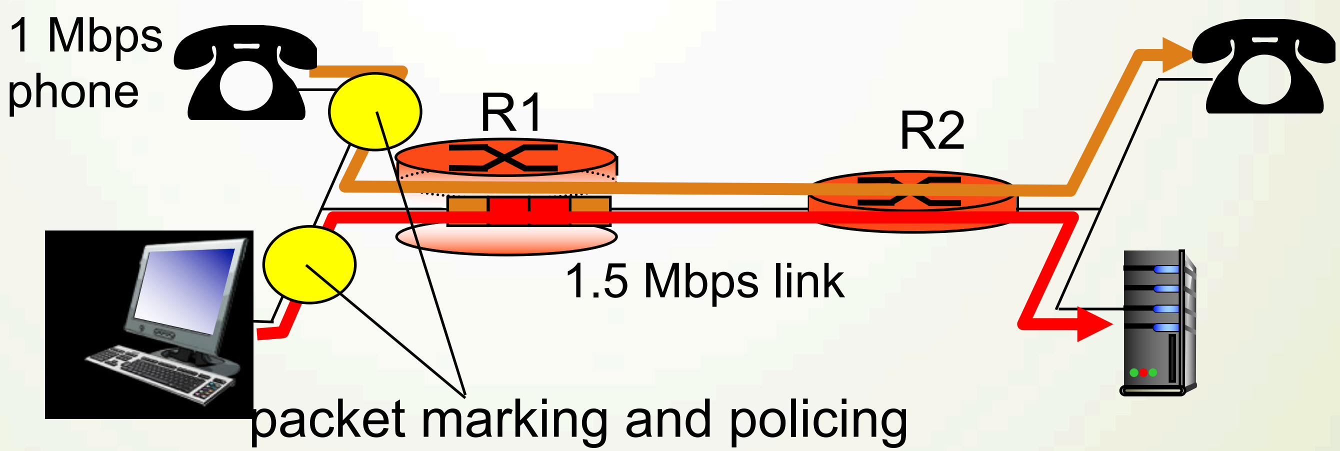
Principle 1: Multiple Classes of Services

- packet marking needed for router to distinguish between different classes; and new router policy to treat packets accordingly



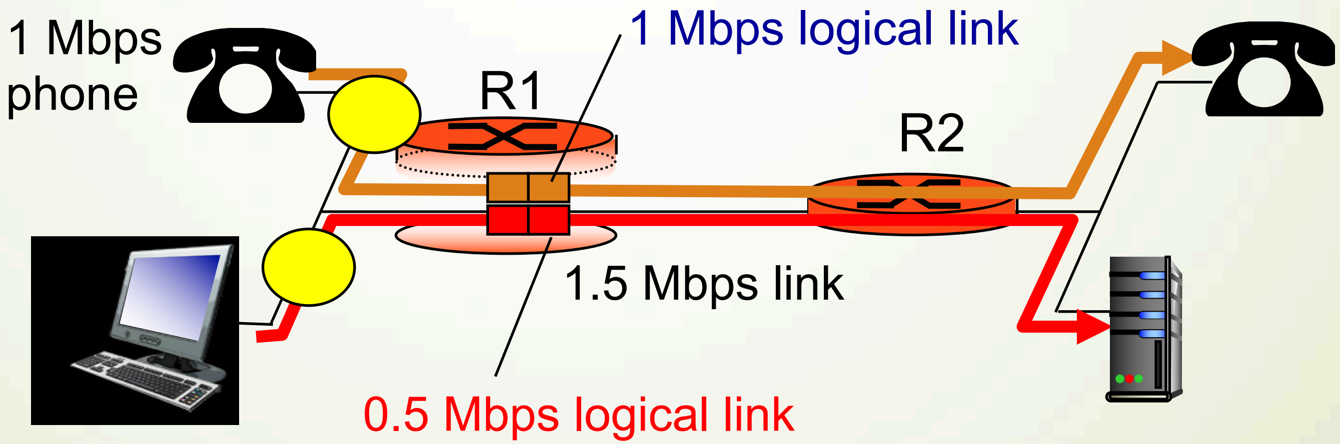
Principle 2: Marking and Policing at the Edge

- provide protection (isolation) for one class from others



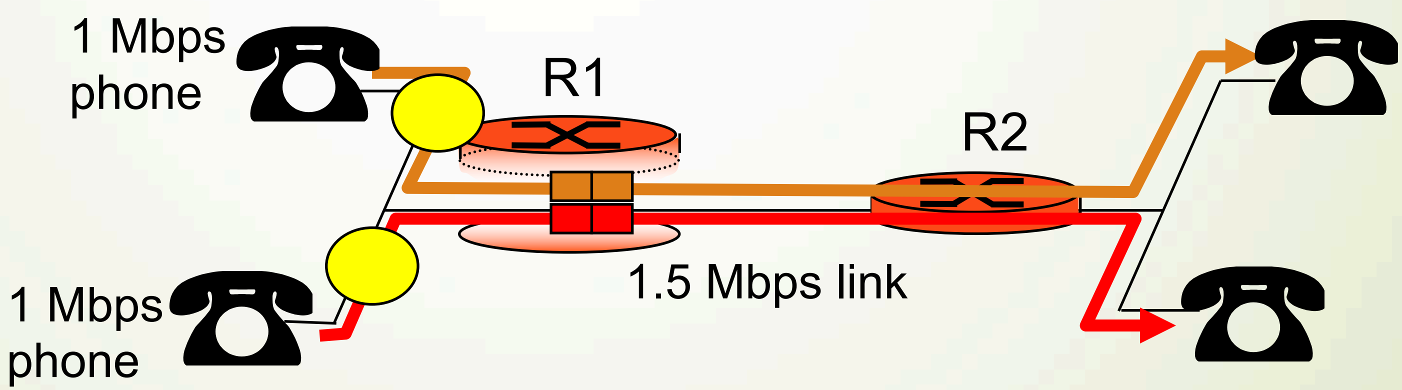
Principle 3: Efficient Usage of Left-Over Bandwidth

- while providing isolation, it is desirable to use resources as efficiently as possible



Principle 4: Can Not Go Beyond Link Capacity

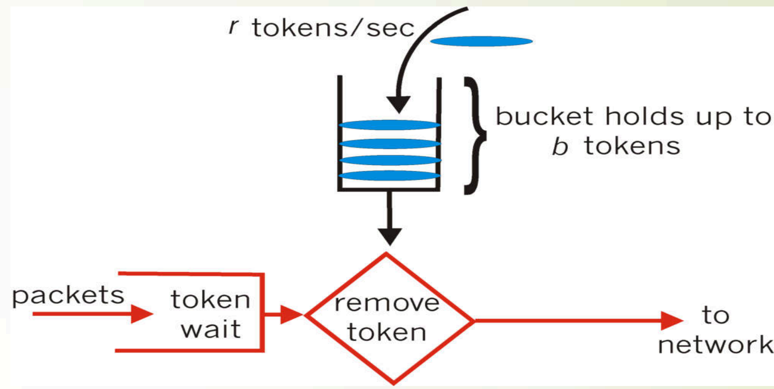
- Admission control: flow declares its needs, network may block call (e.g., busy signal) if it cannot meet needs



Sample Mechanism: Policing

- goal: limit traffic to not exceed declared parameters
- Three common-used metrics:
  - average rate: how many pkts can be sent per unit time (in the long run)
  - peak rate: e.g., 6000 pkts per min (ppm) avg.; 1500 ppm peak rate
  - maximal burst size: max number of pkts sent consecutively (with no intervening idle)

Sample Implementation: Policing



- Token bucket:
  - bucket can hold *b* tokens
  - tokens generated at rate *r* token/sec unless bucket full
  - over interval of length *t*: number of packets admitted less than or equal to $(r\,t + b)$
- Q1: What does a token bucket control? Average rate? Peak Rate? Or Burst Size?
- Q2: How do you control all three common metrics?

Questions

chsu@cs.nthu.edu.tw